Assignment 2: Parallel Programming using CUDA and OpenMP

Graph-Based Local Histogram Equalization on 3D Point Clouds

March 27, 2026

1 Introduction

This report describes the implementation of **local histogram equalization** on 3D point cloud data using CUDA and OpenMP. Each point in the cloud has spatial coordinates (x, y, z) and an intensity value $I \in \{0, 1, \ldots, 255\}$. The task involves computing a local histogram for each point based on its neighborhood, deriving a cumulative distribution function (CDF), and remapping intensities to enhance contrast.

Three different neighborhood-definition strategies are implemented:

1. **Exact KNN** — using a KD-tree for exact k-nearest neighbor search.
2. **Approximate KNN** — using a 3D spatial grid (voxel hashing) for fast approximate neighbor search.
3. **K-Means Clustering** — using parallelized K-Means to group points into clusters.

All three approaches share the same histogram equalization pipeline (local histogram $\rightarrow$ CDF $\rightarrow$ intensity remapping) but differ in how the local neighborhood $\mathcal{N}_k(i)$ is defined.

2 Common Data Structures

2.1 Point Representation

Each point is stored using a 16-byte aligned struct:

```
struct __align__(16) Point {
    int x, y, z, I;
};
```

Listing 1: Point struct with 16-byte alignment

The `__align__(16)` attribute ensures that each `Point` occupies exactly 16 bytes, enabling **coalesced 128-bit memory transactions** on the GPU. When consecutive threads in a warp access consecutive points, the GPU can serve the entire warp with a minimal number of memory transactions, significantly improving global memory throughput.

2.2 Histogram Equalization Formula

For all three methods, once the local histogram $H_i[v]$ is computed, the equalized intensity is:

$$I_i' = \left\lfloor \frac{C_i[I_i] - C_{i,\min}}{m - C_{i,\min}} \times 255 \right\rfloor$$

where $C_i[v] = \sum_{u=0}^{v} H_i[u]$ is the CDF, $C_{i,\min} = \min\{C_i[v] \mid C_i[v] > 0\}$, and m is the neighborhood size (including the query point itself). If $m = C_{i,\min}$, the original intensity is preserved.

3 Approach 1: Exact KNN

3.1 Overview

The exact KNN approach finds the k nearest neighbors for every point using Euclidean distance. A KD-tree is constructed on the CPU and traversed on the GPU in parallel.

3.2 Implementation Details

3.2.1 KD-Tree Construction (Host)

The KD-tree is built recursively on the host using `std::nth_element` for $O(n)$ median selection at each level. Each internal node stores:

```
struct TreeNode {
    int pt_idx;         // Index of the point at this node
    int left, right;    // Child indices (-1 if absent)
    int axis;           // Splitting axis (0=x, 1=y, 2=z)
};
```

Listing 2: KD-tree node structure

The splitting axis cycles through $x \to y \to z$ with depth: `axis` = `depth` mod 3. The tree is stored as a flat array (`std::vector<TreeNode>`) for efficient GPU transfer.

3.2.2 GPU KD-Tree Traversal

Each GPU thread independently traverses the KD-tree for one query point. Key design decisions:

- **Iterative traversal with explicit stack**: A fixed-size stack of depth 32 replaces recursion, which is not efficient on GPUs. Depth 32 is sufficient since $\log_2(n)$ rarely exceeds this for practical input sizes.
- **Max-heap for k-best tracking**: Each thread maintains a max-heap of size k in registers/local memory. The heap root always holds the *worst* (farthest) neighbor, enabling $O(1)$ pruning decisions.
- **Branch pruning**: After visiting the "near" child (the side of the splitting plane where the query lies), the "far" child is only visited if:

$$\texttt{heap_sz} < k \quad \text{or} \quad (\text{axis_dist})^2 \leq \text{heap}[0].\text{dist2}$$

  This prunes large portions of the tree, reducing the average work per query from $O(n)$ to approximately $O(\log n)$.
- **Deterministic tie-breaking**: When two points are equidistant, ties are broken lexicographically by (x, y, z) coordinates to ensure reproducible results.

3.2.3 Integrated Equalization

After KNN search, each thread immediately computes the local histogram, CDF, and equalized intensity in the `equalize()` device function. This avoids a separate kernel launch and the need to store all $n \times k$ neighbor indices in global memory.

```
__device__ __forceinline__
int equalize(const Point* pts, const int* neighbors,
             int k, int self_I) {
    uint8_t hist[HIST_BINS] = {};
    hist[self_I]++;
    for (int j = 0; j < k; j++)
        ++hist[pts[neighbors[j]].I];
    int total = k + 1;
    int cumsum = 0, cdf_min = 0, cdf_self = 0;
    for (int v = 0; v < HIST_BINS; v++) {
        cumsum += hist[v];
        if (cdf_min == 0 && cumsum > 0) cdf_min = cumsum;
        if (v == self_I) cdf_self = cumsum;
    }
    if (total == cdf_min) return self_I;
    int val = (int)(((long long)(cdf_self - cdf_min) * 255)
                    / (total - cdf_min));
    return max(0, min(255, val));
}
```

Listing 3: Inline equalization after KNN search

3.3 Optimizations

- **L1 cache preference**: `cudaFuncCachePreferL1` is set for the KNN kernel since the KD-tree traversal is pointer-chasing heavy and benefits from larger L1 cache for random global memory accesses.
- **Block size of 128**: A smaller block size (`KNN_BLOCK_SZ = 128`) is used because each thread has high register pressure (heap array, stack), allowing more threads per SM.
- **Pinned host memory**: `cudaMallocHost` is used for the point array to enable faster PCIe transfers.

4 Approach 2: Approximate KNN

4.1 Overview

The approximate KNN approach trades exactness for speed by using a **3D spatial grid (voxel hashing)** to restrict the neighbor search to nearby cells. A key design choice is the use of a **precomputed, distance-sorted cell-offset table** stored in constant memory, which replaces the traditional nested-loop expanding-shell pattern with a flat iteration and enables fine-grained cell-level distance pruning.

4.2 Algorithm

1. **Bounding box**: Compute the axis-aligned bounding box via OpenMP parallel reduction.
2. **Grid construction**: Divide 3D space into a uniform voxel grid with adaptive cell size:

$$v_s = \frac{\max(l_x, l_y, l_z)}{\sqrt[3]{27n/(k+1)}}$$

This targets roughly $k{+}1$ points in a $3 \times 3 \times 3$ neighborhood.

3. **Counting sort**: Points are assigned to cells (OpenMP parallel) and reordered by cell ID using an $O(n)$ counting sort. Prefix sums record each cell's start offset and count.
4. **Offset table**: All $(2R{+}1)^3 = 729$ cell offsets (for $R{=}4$) are precomputed, annotated with a *lower-bound distance key*, sorted, and uploaded to `__constant__` memory.
5. **GPU kernel**: Each thread iterates over the sorted offset table, visits the corresponding cell, and maintains a k-best candidate list. The iteration terminates early when the lower-bound key guarantees no closer neighbor exists.

4.3 Implementation Details

4.3.1 Counting Sort (Host)

The original approach used `qsort` ($O(n \log n)$) to reorder points by cell ID. Since cell IDs are bounded integers in $[0, G_x G_y G_z)$, a **counting sort** achieves $O(n + C)$ time where C is the number of cells:

1. Histogram cell occupancies in a single pass.
2. Exclusive prefix sum to obtain write offsets.
3. Scatter points into sorted order using per-cell write cursors.

This eliminates the comparison-based sorting bottleneck entirely.

4.3.2 Precomputed Offset Table

For radius $R = 4$, the $9^3 = 729$ cell offsets $(\Delta x, \Delta y, \Delta z)$ are precomputed on the host. Each offset is annotated with a **lower-bound key**:

$$\mathrm{lb}(\Delta x, \Delta y, \Delta z) = \big(\max(0, |\Delta x|{-}1)\big)^2 + \big(\max(0, |\Delta y|{-}1)\big)^2 + \big(\max(0, |\Delta z|{-}1)\big)^2$$

This key, when multiplied by v_s^2, gives a valid *lower bound* on the squared Euclidean distance from the query point to any point in the target cell. The offsets are sorted by this key, with ties broken by L1 norm, and uploaded to constant memory:

```
struct CellOff { short dx, dy, dz, lb; };
__constant__ CellOff g_cell_offs[729];

// Host: sort by lower-bound key
std::sort(tbl, tbl + cnt,
    [](const CellOff& a, const CellOff& b) {
        if (a.lb != b.lb) return a.lb < b.lb;
        return (abs(a.dx)+abs(a.dy)+abs(a.dz))
             < (abs(b.dx)+abs(b.dy)+abs(b.dz));
    });
```

Listing 4: Offset table structure and initialization

4.3.3 GPU Kernel with Cell-Level Pruning

The kernel iterates over the sorted offset table in a single flat loop instead of nested shell loops. The key optimization is **cell-level distance pruning**:

```
for (int oidx = 0; oidx < noffs; oidx++) {
    CellOff off = g_cell_offs[oidx];

    // If lb * vs^2 >= worst distance, no cell
    // from here onward can improve the k-best set
    if (cnt == k && (double)off.lb * vs2 >= (double)wD)
        break;

    int gx = cx + off.dx, gy = cy + off.dy, ...
    // unsigned compare folds < 0 and >= G checks
    if ((unsigned)gx >= (unsigned)Gx || ...) continue;
    // ... search points in cell (gx, gy, gz)
}
```

Listing 5: Cell-level pruning via lower-bound key

Since offsets are sorted by lower-bound distance, *all subsequent offsets* have equal or larger lower bounds. Once the pruning condition triggers, the loop terminates. This provides finer-grained early exit than the original radius-level check (`if r >= 2 && cnt == k`).

Additional kernel-level optimizations:

- **`__ldg()` intrinsic**: Used for all read-only integer loads (cell offsets, counts, original indices) to route reads through the texture cache path, improving throughput for scattered access patterns.
- **`uint8_t` histogram**: The local histogram uses `uint8_t` instead of `int`, reducing per-thread local memory from 1024 B to 256 B. Since $k \leq 128$, bin counts always fit in a byte.
- **Fused histogram/CDF**: Histogram accumulation, CDF prefix sum, and intensity remapping are performed in a single pass per query point, eliminating the need for a separate 256-element `cdf[]` array.
- **Unsigned boundary checks**: The condition `(unsigned)gx >= (unsigned)Gx` folds both the < 0 and $\geq G$ comparisons into a single branch, reducing instruction count.

4.4 Trade-offs: Accuracy vs. Speed

- **Accuracy**: The search may miss true neighbors beyond the maximum offset radius ($R = 4$). Accuracy depends on the voxel size, point distribution, and k. The adaptive v_s formula ensures that a radius-1 shell typically captures $\sim k$ points; the extended radius provides a margin for non-uniform distributions.
- **Speed**: Per-point work is $O(c)$ where c is the number of candidate points in visited cells. The cell-level pruning ensures that c is minimized by visiting only cells whose lower-bound distance is competitive with the current k-best set. The counting sort reduces host preprocessing from $O(n \log n)$ to $O(n)$.
- **Constant memory efficiency**: The 729-entry offset table consumes $729 \times 8 = 5{,}832$ bytes of constant memory. Since all threads read the same offsets in lockstep, the constant cache broadcasts each entry to the full warp at zero extra cost.

5 Approach 3: K-Means Clustering

5.1 Overview

In the K-Means approach, the neighborhood of a point is defined as all points belonging to the same cluster. The algorithm performs iterative assignment and centroid update until convergence or T iterations, followed by per-cluster histogram equalization.

5.2 Implementation Details

The K-Means pipeline consists of five GPU kernels:

5.2.1 Kernel 1: Cluster Assignment (`asgn_cluster`)

This is the most performance-critical kernel and uses several optimizations:

- **Constant memory for centroids**: Centroids are stored in `__constant__` memory (`dc_cent[128]`), which is cached and broadcast to all threads in a warp simultaneously. Since every thread reads the same centroid during distance computation, this exploits the broadcast capability of constant memory.
- **Warp-level shared memory accumulation**: Instead of having every thread atomically update global accumulators, each warp accumulates partial sums for centroid coordinates (x, y, z) and counts in shared memory. After the block completes, partial sums are reduced across warps and then atomically added to global accumulators. This reduces the number of global atomics by a factor of 32 (warp size).
- **Convergence detection**: An `atomicOr` on a global flag detects whether any point changed its assignment, enabling early termination.
- **Deterministic tie-breaking**: When two centroids are equidistant, the one with lexicographically smaller (x, y, z) coordinates is chosen.

```
extern __shared__ int smem[];
int* sx = smem;                          // x sums per warp
int* sy = smem + n_warp * k;             // y sums per warp
int* sz = smem + 2 * n_warp * k;         // z sums per warp
int* sc = smem + 3 * n_warp * k;         // counts per warp
```

Listing 6: Shared memory layout for warp-level accumulation

5.2.2 Kernel 2: Centroid Update (`km_update`)

A simple kernel that divides accumulated sums by counts to compute new centroids. One thread per cluster.

5.2.3 Kernel 3: Cluster Histogram (`km_hist`)

This kernel uses **warp-level primitives** for efficient histogram construction:

- `__ballot_sync`: Creates a bitmask of threads with the same (cluster, intensity) key.
- `__shfl_sync`: Broadcasts keys across the warp for comparison.
- `__popc`: Counts the number of matching threads (population count).

Instead of each thread issuing an individual `atomicAdd`, threads with the same key are grouped and a single `atomicAdd` is performed with the aggregated count. This reduces atomic contention by up to 32×.

```
unsigned remaining = mask;
while (remaining) {
    int src = __ffs(remaining) - 1;
    int src_key = __shfl_sync(mask, key, src);
    unsigned same = __ballot_sync(mask, key == src_key);
    if (lane == src)
        atomicAdd(&cluster_hist[cluster*256 + intensity],
                  __popc(same));
    remaining &= ~same;
}
```

Listing 7: Warp-level histogram aggregation

5.2.4 Kernel 4: CDF Computation (`cdf`)

One thread per cluster computes the prefix sum over the 256-bin histogram. Since k (number of clusters) is typically small (≤ 128), this kernel launches few threads but each does a serial scan of 256 elements, which is efficient.

5.2.5 Kernel 5: Intensity Remapping (`km_equalize`)

Each thread reads its cluster assignment, looks up the precomputed CDF and $C_{\min}$, and applies the equalization formula. Fully parallel with no inter-thread dependencies.

5.3 Optimizations

- **Shared memory cache preference**: `cudaFuncCachePreferShared` is set for the assignment kernel since it makes heavy use of shared memory for warp-level accumulation.
- **Asynchronous memsets**: `cudaMemsetAsync` is used to overlap zeroing of accumulators with kernel execution.
- **Pinned memory**: Centroids use `cudaMallocHost` for faster host-device transfers during the iterative loop.

6 Runtime Analysis

6.1 Computational Complexity

Table **??** summarizes the theoretical per-point complexity of each approach.

Table 1: Asymptotic per-point complexity comparison

Phase	Exact KNN	Approx KNN	K-Means
Preprocessing	$O(n \log n)$ (tree build)	$O(n \log n)$ (sort)	—
Neighbor Search	$O(\log n)$ avg	$O(c)$	$O(k)$ per iter $\times T$
Equalization	$O(k + 256)$	$O(k + 256)$	$O(256)$

Here c denotes the average number of candidate points examined in the approximate KNN grid cells, and T is the number of K-Means iterations.

6.2 Expected Performance Characteristics

- **Exact KNN**: The KD-tree traversal involves *pointer-chasing* (random memory accesses following tree pointers), which is inherently unfriendly to GPU architectures. Each thread may follow a different path through the tree, leading to **warp divergence**. However, the aggressive pruning of the "far" subtree keeps the average number of nodes visited manageable.
- **Approximate KNN**: The grid-based approach exhibits better **memory locality** — points in the same cell are contiguous in memory due to the sorting step. Threads processing nearby points tend to access the same cells, benefiting from the L1/L2 cache. The fixed maximum radius (4) bounds the worst-case work per thread, reducing warp divergence compared to KD-tree traversal.
- **K-Means**: The assignment kernel has the most **regular memory access pattern** — all threads read the same centroids (broadcast via constant memory) and write to a small number of accumulators. The histogram and equalization are performed as a single pass after clustering converges. The total runtime depends heavily on the number of iterations T needed for convergence.

6.3 Comparative Summary

Table 2: Qualitative comparison of the three approaches

Property	**Exact KNN**	**Approx KNN**	**K-Means**
Accuracy	Exact	Approximate	Different model
Memory locality	Poor (tree)	Good (sorted grid)	Excellent (broadcast)
Warp divergence	High	Moderate	Low
GPU parallelism	Good	Excellent	Excellent
Preprocessing cost	KD-tree build	Sort + grid	None

The approximate KNN is expected to provide the best practical speedup over exact KNN due to its bounded search space and better memory access patterns. K-Means benefits from the most regular computation but produces a fundamentally different neighborhood definition (cluster-based vs. distance-based).

7 OpenMP Usage

OpenMP is used for host-side preprocessing to minimize CPU bottlenecks:

- **Bounding box computation**: `#pragma omp parallel for reduction(min/max:...)` computes the spatial extents in parallel.
- **Cell assignment**: Mapping points to grid cells is embarrassingly parallel and uses `#pragma omp parallel for`.
- **Data reordering**: After sorting, the permuted point and original-index arrays are constructed in parallel.
- **Timing**: `omp_get_wtime()` provides wall-clock timing for performance measurement.

8 Conclusion

This report presented three parallelized approaches to local histogram equalization on 3D point clouds using CUDA and OpenMP. The exact KNN method uses a KD-tree for guaranteed correctness but suffers from irregular memory access patterns on the GPU. The approximate KNN method achieves significant speedup by trading exact correctness for spatial locality through grid-based search, while still producing results close to the ground truth. The K-Means approach provides the most GPU-friendly computation pattern by defining neighborhoods as clusters, leveraging constant memory broadcasts and warp-level primitives for efficient histogram construction.

Each approach demonstrates different trade-offs in the parallelization of spatial queries on GPU architectures — from the irregular, pointer-chasing nature of tree traversals to the highly regular, broadcast-friendly pattern of centroid-based assignment.